

How Modern Browsers Work

Addy Osmani

June 21, 2026

Web developers often treat the browser as a black box that magically transforms HTML, CSS, and JavaScript into interactive web applications. This article dives into how modern browsers work — focusing on Chromium's architecture and internals, while noting where other engines differ. From the networking stack and parsing pipeline to the rendering process via Blink, JavaScript engine via V8, module loading, multi-process architecture, security sandboxing, and developer tooling.

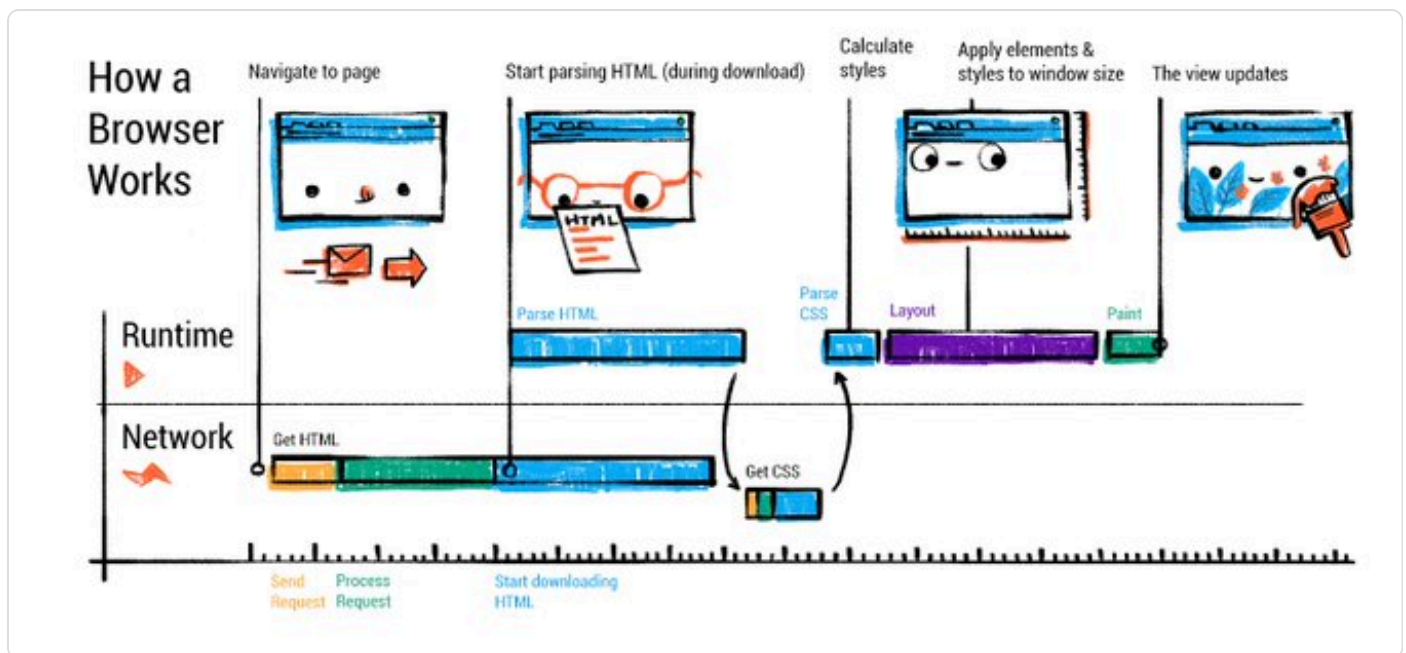


How modern
browsers work

Original: x.com/addyosmani/article/2068394292796871019

Illustrations by Susie Lu

1. Networking and Resource Loading



Networking and resource loading pipeline — Illustration by Susie Lu

Every page load begins with the browser's networking stack fetching resources from the web. When you enter a URL or click a link, the **browser's UI thread (running in the "browser process")** kicks off a navigation request.

*The **browser process** is the main, controlling process that manages all other processes and the browser's user interface. Everything that happens outside of a specific web page tab is controlled by the browser process.*

The steps include:

Some free text>>>>

URL parsing and security checks: The browser parses the URL to determine the scheme (http, https, etc.) and target domain. It also decides if the input is a search query or URL (in Chrome's omnibox, for example). Security features like blocklists may be checked here to avoid phishing sites.

DNS lookup: The network stack resolves the domain name to an IP address (unless it's cached). This may involve contacting a DNS server. Modern browsers might use OS DNS services or even DNS over HTTPS (DoH) if configured, but ultimately they obtain an IP for the host.

Establishing a connection: If no open connection to the server exists, the browser opens one. For HTTPS URLs, this includes a TLS handshake to securely exchange keys and verify certificates. The browser's network thread handles protocols like TCP/TLS setup transparently.

Sending the HTTP request: Once connected, an HTTP GET request (or other method) is sent for the resource. Browsers today default to HTTP/2 or HTTP/3 if the server supports it, which allows multiplexing multiple resource requests over one connection. This improves performance by avoiding the old limit of ~6 parallel connections per host (HTTP/1.1). For example, with HTTP/2 the HTML, CSS, JS, images can all be fetched concurrently on one TCP/TLS link, and with HTTP/3 (over QUIC UDP) setup latency is further reduced.

Receiving the response: The server responds with an HTTP status and headers, followed by the response body. The browser reads the response stream. It may need to sniff the MIME type if the Content-Type header is missing or incorrect, to decide how to handle the content. There are security measures here too: the network layer checks Content-Type and may block suspicious MIME mismatches or disallowed cross-origin data (Chrome's CORB — Cross-Origin Read Blocking — is one such mechanism). The browser also consults Safe Browsing or similar services to block known malicious payloads.

Redirects and next steps: If the response is an HTTP redirect (e.g. 301 or 302 with a Location header), the network code will follow the redirect and repeat the request to the new URL. Only once a final response with actual content is obtained does the browser move on to processing that content.

All these steps happen in the network stack, which in Chromium is run in a dedicated Network Service (now typically a separate process, as part of Chrome's "servicification" effort). Importantly, this design means the renderer (which will execute the page's code) doesn't directly access the network — it asks the browser process to fetch what it needs, a security win.

Speculative Loading and Resource Optimization

Modern browsers implement sophisticated performance optimizations in the networking stage. Chrome will proactively perform a DNS prefetch or open a TCP connection when you hover over a link or start typing a URL so that if you click, some latency is already shaved off. There's also HTTP caching: the network stack can satisfy requests from the browser cache if the resource is cached and fresh, avoiding a network trip.

Preload scanner operation: Chromium implements a sophisticated preload scanner that tokenizes HTML markup ahead of the main parser. When the primary HTML parser is blocked by CSS or synchronous JavaScript, the preload scanner continues examining the raw markup to identify resources like images, scripts, and stylesheets that can be fetched in parallel. The preload scanner cannot discover resources injected via JavaScript, making such resources likely to be loaded consecutively rather than concurrently.

Early Hints (HTTP 103): Early Hints allows servers to send resource hints while generating the main response, using HTTP 103 status codes. This enables preconnect and preload hints to be sent during server think-time, potentially improving Largest Contentful Paint by several hundred milliseconds. Early Hints are only available for navigation requests and support preconnect and preload directives, but not prefetch.

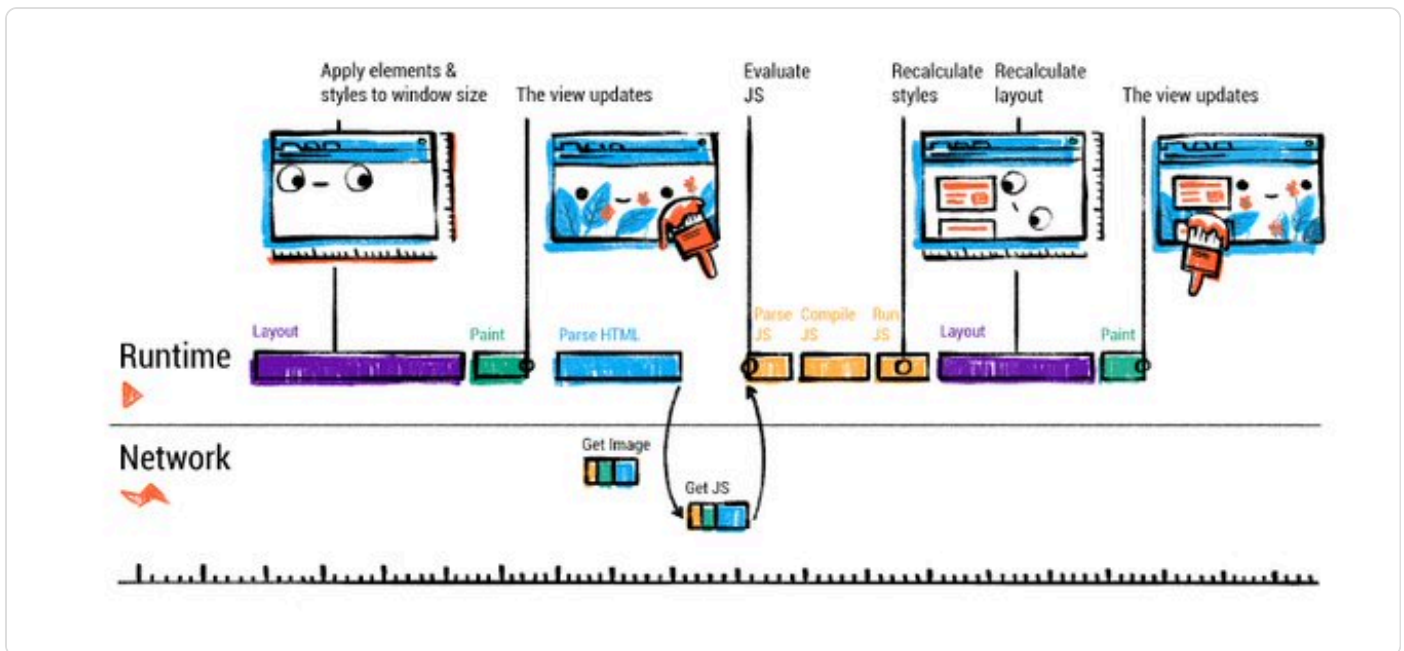
Speculation Rules API: A recent web standard that allows defining rules to dynamically prefetch and prerender URLs based on user interaction patterns. Unlike traditional link prefetch, this API can prerender entire pages including JavaScript execution, leading to near-instant load times. The API uses JSON syntax within script elements or HTTP headers to specify which URLs should be speculatively loaded. Chrome has limits to prevent overuse, with different capacity settings based on urgency levels.

HTTP/2 and HTTP/3: Most Chromium-based browsers and Firefox support HTTP/2 fully, and HTTP/3 (based on QUIC) is also widely supported (Chrome has it enabled by default for supporting sites). These protocols improve page load by allowing concurrent transfers and reducing handshake overhead. From a developer perspective, this means you may no longer need sprite sheets or domain sharding tricks — the browser can efficiently fetch many small files in parallel on one connection.

Resource prioritization: The browser also prioritizes certain resources over others. Typically, HTML and CSS are high priority (as they block rendering), scripts might be medium, and images maybe lower. Chromium's network stack assigns weights and can even cancel or defer requests to prioritize what's needed for an initial render. Developers can use `link rel=preload` and `Fetch Priority` to influence resource prioritization.

By the end of the networking phase, the browser has the initial HTML for the page. At this point, Chrome's browser process chooses a renderer process to handle the content. Chrome will often launch a new renderer process in parallel with the network request (speculatively) so that it's ready to go when the data arrives. Once the response is fully received (or as it streams in), the browser process commits the navigation: it signals the renderer process to take the stream of bytes and start processing the page.

2. Parsing HTML, CSS, and JavaScript



HTML parsing and DOM construction — Illustration by Susie Lu

When the renderer process receives the HTML content, its main thread begins to parse it according to the HTML specification. The result of parsing HTML is the DOM (Document Object Model) — a tree of objects representing the page structure. Parsing is incremental and can interleave with network reading (browsers parse HTML in a streaming fashion, so the DOM can start being built even before the entire HTML file is downloaded).

HTML Parsing and DOM Construction

HTML parsing is defined by the HTML Standard as an error-tolerant process that will produce a DOM no matter how malformed the markup is. This means even if you forget a closing `</p>` tag or have nested tags incorrectly, the parser will implicitly fix or adjust the DOM tree so that it's valid. For example, `<p>Hello<div>World</div>` will automatically end the `<p>` before the `<div>` in the DOM structure.

One important aspect is that the HTML parser may encounter resources to fetch as it goes: encountering a `<link rel="stylesheet">` will prompt the browser to request the CSS file, and encountering an `<img>` will trigger an image request. These happen in parallel to parsing, with one big exception: scripts.

Handling `<script>` Tags

If the HTML parser comes across a `<script>` tag, it pauses parsing and must execute the script before continuing (by default). This is because scripts can use `document.write()` or other DOM manipulation that can alter the page structure. This script execution blocking behavior is why including large `<script>`

files in the head can slow down page rendering.

However, developers can modify this behavior with attributes: adding `defer` or `async` to a `<script>` tag changes how the browser handles it. With **async**, the script file is fetched in parallel and executed as soon as it's ready, without pausing HTML parsing. With **defer**, the script is fetched in parallel but execution is deferred until the HTML parsing is done (and will execute in the original order). ES6 modules (using `<script type="module">`) are automatically deferred as well.

CSS Parsing and the CSSOM

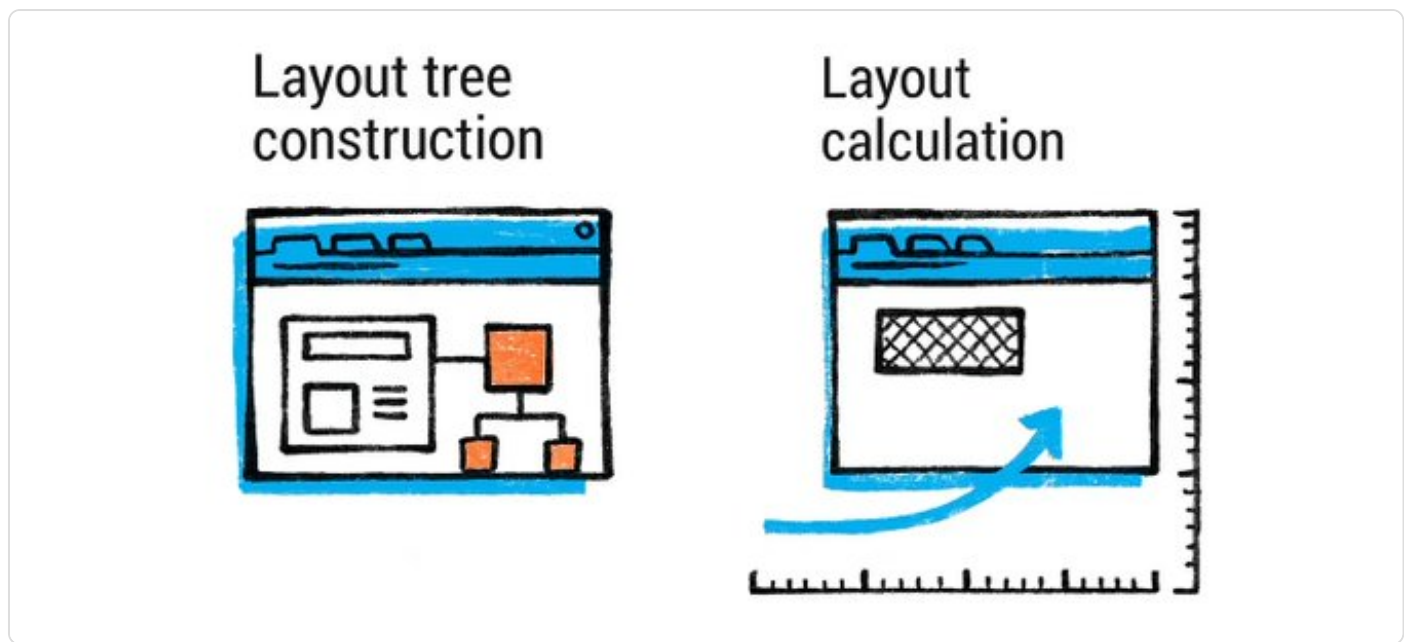
Alongside HTML, CSS text must be parsed into a structure the browser can work with — often called the CSSOM (CSS Object Model). The CSSOM is essentially a representation of all the styles that apply to the document. The browser's CSS parser reads CSS files (or `<style>` blocks) and turns them into a list of CSS rules. Then, as the DOM is being constructed, the browser will compute the style for each DOM node. This step is usually called **style resolution** or **style calculation**. The browser combines the DOM and CSSOM to determine, for each element, which CSS rules apply and what the final computed styles are.

Even without any author CSS, every element has default browser styles (the user-agent stylesheet). The style calculation step uses all applicable styles (user agent, user styles, author styles) to finalize each element's styling.

Render-Blocking Behavior

While HTML parsing can proceed without CSS fully loaded, there is a render-blocking relationship: browsers typically wait to perform the first render until CSS is loaded (for CSS in the `<head>`). This is because applying an incomplete stylesheet could flash unstyled content. As a rule of thumb, put stylesheet links in the head (they block rendering but are needed early) and put non-critical or large scripts with `defer` / `async` or at the bottom so they don't delay DOM parsing.

3. Styling and Layout



Styling and layout — Illustration by Susie Lu

At this stage, the browser's renderer process knows the structure of the DOM and each element's computed style. The next question is: where do all these elements go on the screen? This is the job of **layout** (also known as "reflow" or "layout calculation"). The browser calculates the geometry of each element — their size and position — according to the CSS rules and the DOM hierarchy.

Layout Tree Construction

The browser walks the DOM tree and generates a layout tree (sometimes called the render tree or frame tree). The layout tree is similar to the DOM tree in structure, but it omits non-visual elements (e.g. script or meta tags don't produce boxes) and may split some elements into multiple boxes if needed.

During layout, the browser computes the exact position (x, y coordinates) and size (width, height) for each element's box. This involves algorithms defined by CSS specifications: in a normal document flow, block-level elements stack top-to-bottom, whereas inline elements flow within lines. Modern layout modes like flexbox or grid have their own algorithms.

Some details about the layout tree:

- Elements with `display:none` are omitted entirely from the layout tree. In contrast, elements with `visibility:hidden` do get a layout box (taking up space), just not painted later.
- Pseudo-elements like `::before` or `::after` that generate content are included in the layout tree.
- The layout tree nodes know their geometry — position relative to the viewport and dimensions.

Layout Calculation

Layout is typically a recursive process. Starting from the root (`<html>` element), the browser computes the size of the viewport and then lays out child elements within it. Many elements' sizes depend on their children or parent. The layout algorithm often has to do multiple passes for things like floats or certain complex interactions.

Layout can be an expensive operation, especially if done repeatedly. If JavaScript later changes the size of an element or adds content, it can force a relayout of some or all of the page. Developers often hear advice about avoiding **layout thrashing** — reading layout info in JS right after modifying DOM, which can force synchronous recalculation.

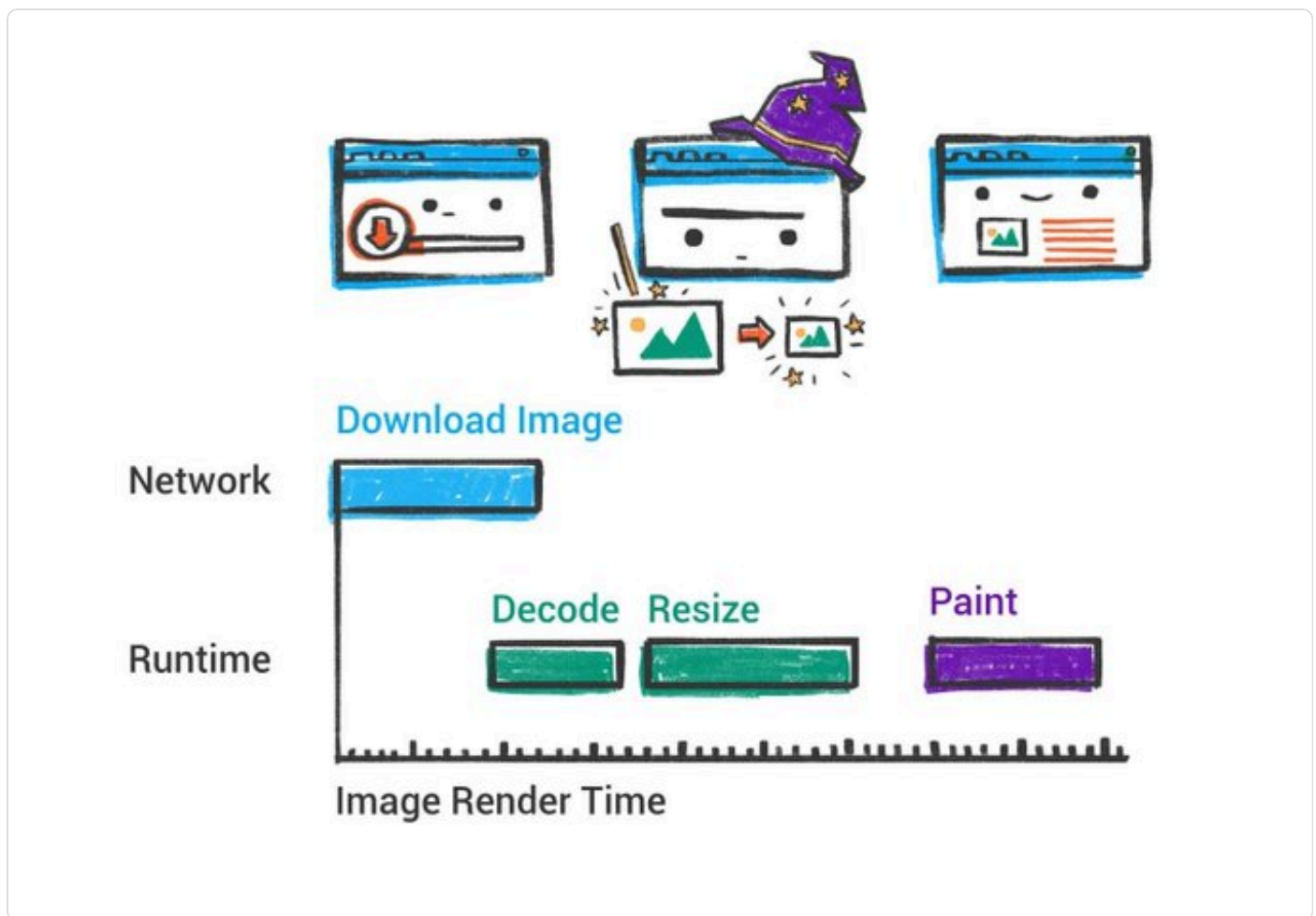
Style and Layout Recap

From HTML and CSS the browser builds:

- **DOM tree** — structure and content
- **CSSOM** — parsed CSS rules
- **Computed Styles** — the result of matching CSS rules to each DOM node
- **Layout tree** — DOM tree filtered to visual elements, with geometry for each node

Each stage builds on the last. If any stage changes (e.g. if a script alters the DOM or modifies a CSS property), the subsequent stages may need to update.

4. Painting, Compositing, and GPU Rendering



Painting, compositing, and GPU rendering — Illustration by Susie Lu

Painting is the process of taking the structured layout information and actually producing pixels on the screen. Modern browsers split the work into multiple stages and leverage the GPU for efficiency.

Painting / Rasterization

On the renderer's main thread, after layout, Chrome generates paint records (or a display list) by walking the layout tree. This is basically a list of drawing operations with their coordinates — "draw rect at (x,y) with width W and height H and fill color blue, then draw text 'Hello' at (x2,y2) with font XYZ..." and so on. This list is in the correct z-index order so that overlapping elements paint correctly.

Layering and Compositing

Compositing is an optimization where the page is split into several layers that can be handled independently. A positioned element with a CSS transform or an animation might get its own layer. Layers are like separate "scratch canvases" — the browser can rasterize each layer separately, and then the compositor can

blend them together on the screen, often using the GPU.

After determining layers, Chrome's main thread hands off to the **compositor thread**. The compositor thread runs in the renderer process but separate from the main thread (so it can keep working even if the main JS thread is busy, which is great for smooth scrolling and animations).

Rasterization with GPU Assistance

The compositor thread breaks layers into smaller tiles (256×256 or 512×512 pixel chunks). It then dispatches these to several raster worker threads for concurrent rasterization. Each raster worker takes a tile and produces a bitmap. Skia (Chrome's graphics library) can use the CPU or GPU to rasterize. The rasterized tiles are stored in GPU memory as textures.

The compositor then assembles a compositor frame — basically a message to the browser process that includes all the quads (tiles of layers) that make up the screen. The GPU process receives the frames and uses the GPU (via OpenGL/DirectX/Metal) to composite them — drawing each texture at the right place on screen.

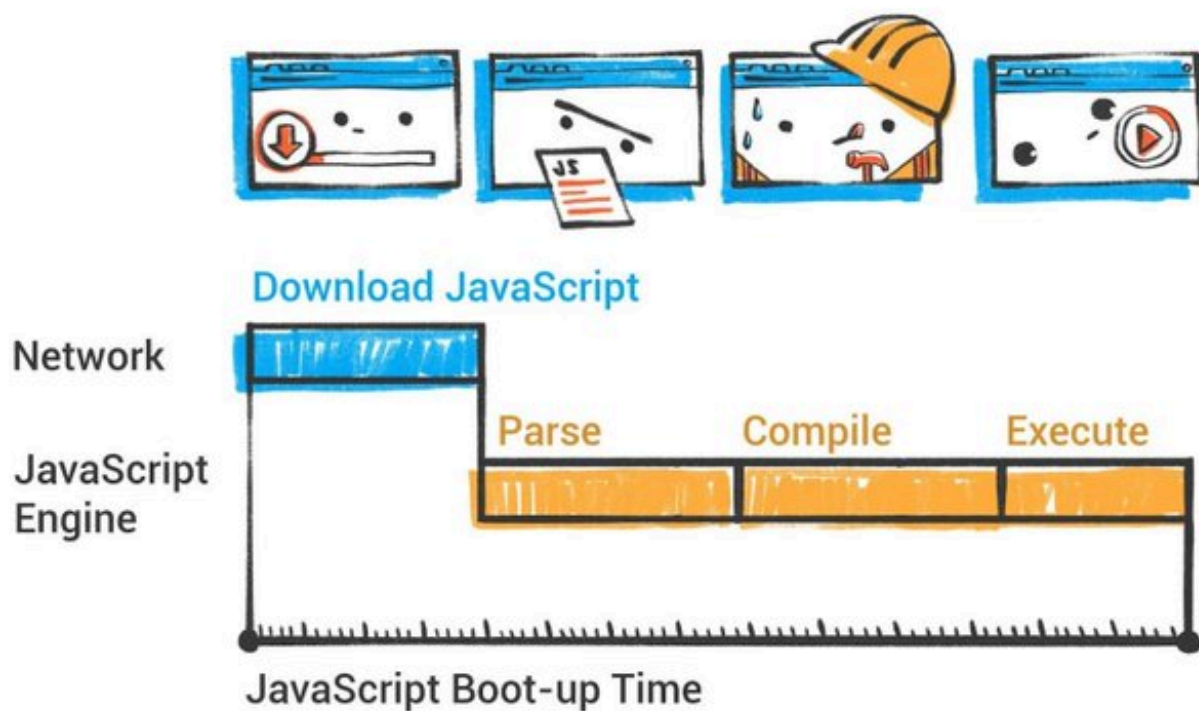
Why This Pipeline Matters

The advantage is apparent when you scroll or animate. Scrolling mostly just changes the viewport on a larger page texture — the compositor can shift layer positions without the main thread having to repaint everything. Animations that are "compositing-only" (changing `transform` or `opacity`) can run at 60 FPS smoothly even if the main thread is busy. Animating something like `height` or `background-color` forces re-layout or re-paint each frame, which causes jank.

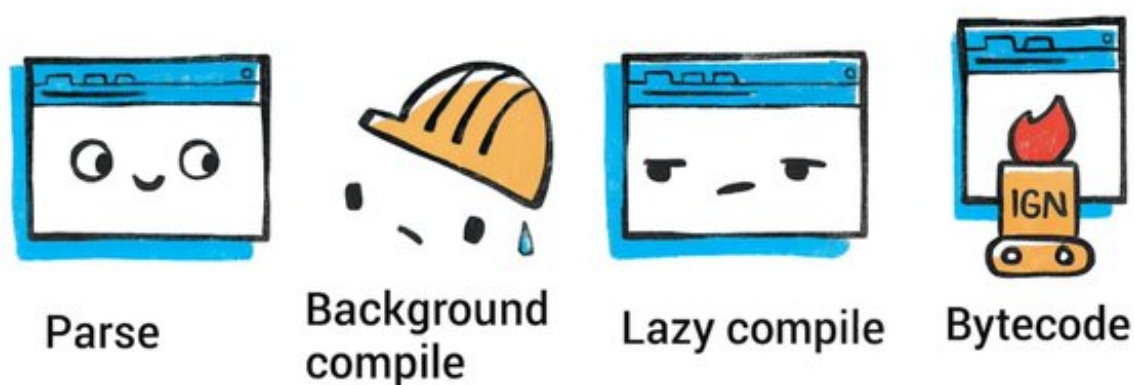
DOM → Style → Layout → Paint → Layerize → Raster → Composite (GPU)

In Chromium, the GPU process is a separate process whose job is to interface with the graphics hardware. By isolating it in a process, a buggy GPU driver that crashes won't take down the whole browser. For each animation frame (target 60fps or 16.7ms per frame), the compositor aims to produce a frame. Techniques like `requestAnimationFrame` align JS updates to frame boundaries to help with smooth rendering.

5. Inside the JavaScript Engine (V8)



Inside the V8 JavaScript engine — Illustration by Susie Lu



V8 parsing and compilation pipeline — Illustration by Susie Lu

In Chromium browsers, the V8 engine executes JavaScript (and WebAssembly). Understanding V8's workings can help developers write performant JS.

Background Compilation

Starting with Chrome 66, V8 compiles JavaScript source code on a background thread, reducing the amount of time spent compiling on the main thread by between 5% to 20% on typical websites. V8 can start parsing JavaScript source code as soon as the first chunk is downloaded from the network and continue parsing in parallel while streaming the file. Almost all script compilation occurs on background threads, with only short AST internalization and bytecode finalization steps happening on the main thread.

Parsing and Bytecode

When a `<script>` is encountered, V8 first parses the JavaScript source code, producing an Abstract Syntax Tree (AST) representation. The **preparser** is a copy of the parser that does the bare minimum needed to skip over functions — it verifies that functions are syntactically valid and produces all information needed for outer functions to be compiled correctly. When a preparsed function is later called, it is fully parsed and compiled on-demand.

Rather than interpreting directly from the AST, V8 uses a bytecode interpreter called **Ignition** (introduced in 2016). Ignition compiles the JavaScript into a compact bytecode format. This initial compilation is quite fast and the goal is to start executing the code quickly with minimal upfront cost.

Explicit Compile Hints: V8 has introduced a feature that allows developers to instruct V8 to parse and compile code immediately on load through eager compilation. Files with this hint are compiled on background threads. Experiments showed performance improvements in 17 out of 20 cases, with an average 630ms reduction in foreground parse and compile times.

JIT Compilation Tiers

V8 employs multiple tiers of Just-In-Time compilers to accelerate hot code. The idea is to spend more compilation effort on code that runs a lot, to make it faster, while not wasting time optimizing code that runs only once.

TIER	NAME	ROLE
1	Ignition	Bytecode interpreter. Fast startup, collects profiling data.
2	Sparkplug	Baseline JIT (2021). Quick compilation to native code without heavy optimization.
3	Maglev	Mid-tier optimizer (2023). Code generation $\sim 20\times$ slower than Sparkplug but $10\text{--}100\times$ faster than TurboFan. Bridges the gap for moderately hot functions.
4	TurboFan	Top-tier optimizer. Uses type feedback for highly optimized machine code with inlining, bounds-check elision, etc. Backend being replaced by Turboshift (CFG-based IR).

One challenge with JIT optimization is that JavaScript is dynamically typed. V8 might optimize code under certain assumptions (e.g. a variable is always an integer). If a later call violates those assumptions, V8 performs a **deoptimization**: it falls back to a less optimized version. This mechanism relies on "inline caches" and type feedback to quickly adapt.

Bytecode Flushing and Memory Management

V8 implements bytecode flushing: if a function remains unused after multiple garbage collections, its bytecode will be reclaimed. When executed again, the parser uses previously stored results to regenerate the bytecode more quickly.

Garbage Collection (Orinoco GC)

V8 automatically manages memory using a generational, incremental, and concurrent garbage collector:

- **Generational:** New objects are allocated in the young generation ("nursery"), collected frequently with a fast scavenging algorithm. Survivors get promoted to the old generation.
- **Mark-and-sweep/compact:** For the old generation, V8 marks all reachable objects, then sweeps unreferenced objects. Much of the marking is concurrent — done on a background thread while JS is still running.
- **Incremental GC:** Garbage collection in small slices rather than one big pause, spread out to avoid jank.
- **Parallel GC:** Parts of GC (marking, sweeping) can run on parallel threads on multi-core machines.

V8 and Web APIs: V8 covers the core JavaScript language and runtime, but many "browser APIs" (DOM methods, fetch, etc.) are provided by the browser and exposed to JS via bindings. Chrome uses an IDL to generate efficient bindings.

6. Module Loading and Import Maps

JavaScript modules (ES6 modules) introduce a different loading and execution model compared to classic `<script>` tags. Modules are files that explicitly import/export values.

Static Module Imports

When the browser encounters a `<script type="module" src="main.js">`, it treats `main.js` as a module entry point. The browser will fetch `main.js`, parse it as an ES module, find any `import` statements, and construct a module dependency graph. It will initiate fetching of any imported modules recursively. Only once the entire graph is fetched and parsed can the browser evaluate the modules. Module scripts are deferred by nature — execution happens in dependency order (depth-first, post-order).

Dynamic import()

ES2020 introduced `import(moduleSpecifier)` as an expression, allowing code to load a module on the fly (returning a promise). Under the hood, `import()` triggers the browser to fetch the requested module and its dependencies, then instantiate and execute it. Dynamic import can be used in non-module scripts too.

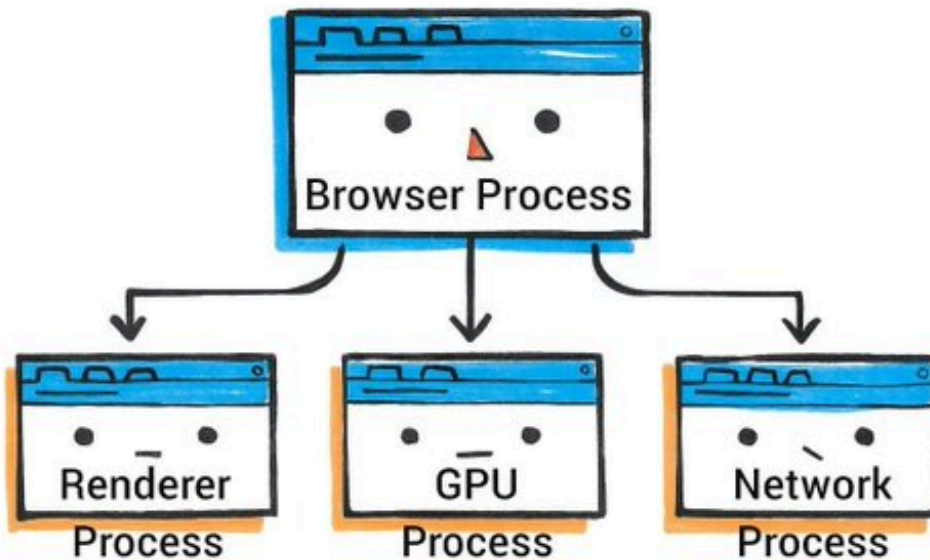
Import Maps

An import map is a JSON configuration (via `<script type="importmap">`) that tells the browser how to resolve module specifiers to real URLs. This allows "bare" specifiers like package names to work on the web by mapping them to CDN URLs or local paths. Since 2023, import maps are supported in all major browsers (Chrome 89+, Firefox 108+, Safari 16.4+).

Once fetched and compiled, module code executes in strict mode and with its own top-level scope. The exports are cached so if another module imports the same module later, it doesn't re-run it.

7. Browser Multi-Process Architecture

Browser Multi-Process Architecture



Browser multi-process architecture — Illustration by Susie Lu

Modern browsers all use a multi-process architecture for stability, security, and performance isolation. Chrome was a pioneer of this approach in 2008.

In Chromium, there is one **Browser Process** that is central — responsible for the UI (address bar, bookmarks, menus) and coordinating high-level tasks. Then, for each tab (and sometimes for each site in a tab), Chrome creates a **Renderer Process** running the Blink rendering engine and V8 JS engine.

Key Processes in Chrome

PROCESS	ROLE
Browser	UI, coordination, navigation, spawns other processes
Renderer	Runs Blink + V8 for each tab/site. Sandboxed.
GPU	Communicates with graphics hardware. Separate so a GPU crash doesn't take down renderers.
Network	Handles network requests, DNS. Part of "servicification."
Utility	Audio playback, image decoding, etc.
Extension	Chrome extensions run isolated from websites.

Benefits of Multi-Process

- **Stability:** If a web page crashes, it doesn't crash the whole browser. Chrome can show "Aw, Snap" for a single tab.
- **Security (Sandboxing):** Even if an attacker finds a vulnerability in the rendering engine, they are trapped in the sandbox — the renderer process cannot read your files or open arbitrary network connections.
- **Performance Isolation:** Intensive work in one tab is confined to that tab's renderer process. Other tabs remain responsive.
- **Memory segmentation:** Each process has its own address space, preventing cross-site data snooping. When a tab is closed, the OS can reclaim all memory efficiently.

Site Isolation

Initially, Chrome's model was one process per tab. Over time they evolved it to one process per site (especially after Spectre). As of 2024, site isolation is enabled by default for 99% of Chrome users across desktop platforms. A tab with `example.com` and an iframe of `evil.com` would put `evil.com`'s iframe in a separate process. Site isolation causes Chrome to use 10–13% more system resources but provides crucial security benefits.

Firefox and Safari

Firefox (Electrolysis/e10s): Uses multiple content processes (by default 8 for web content). With Project Fission (site isolation), Firefox can spin up new processes for cross-site iframes (enabled by default in Firefox 108+). Firefox also has a GPU process (for WebRender) and a separate networking process.

Safari (WebKit2): Each tab's content is in a separate WebContent process and a central UI process controls them. WebContent processes are sandboxed and cannot directly access devices or files.

Servicification

Chromium is splitting browser components into services that can run in separate processes — the Network Service, for example, was made a separate module. Powerful systems can run each service in its own process, whereas constrained devices might consolidate some services back into one process to save overhead.

8. Site Isolation and Sandboxing

Site isolation and sandboxing build on the multi-process foundation to ensure that even if malicious code runs in the browser, it cannot easily steal data from other sites or access your system.

Site Isolation and Spectre

Chrome's site isolation was boosted after the Spectre vulnerability came to light in 2018. Spectre showed that malicious JavaScript could potentially read memory it shouldn't (by exploiting CPU speculative execution). If two sites were in the same process, a malicious site could use Spectre to snoop on the sensitive site's memory. The only robust solution is to not let them share a process at all. Chrome made site isolation a default: every site gets its own process, including cross-origin iframes. Firefox has followed with Project Fission.

Sandboxing

Each renderer process runs in a sandbox with restricted privileges. On Windows, Chrome uses a job object and drops privileges. On Linux, it uses namespaces and seccomp filters. The renderer can compute and render content but if it tries to open a file, camera, or microphone, it will be blocked unless going through proper channels that ask user permission via the browser process.

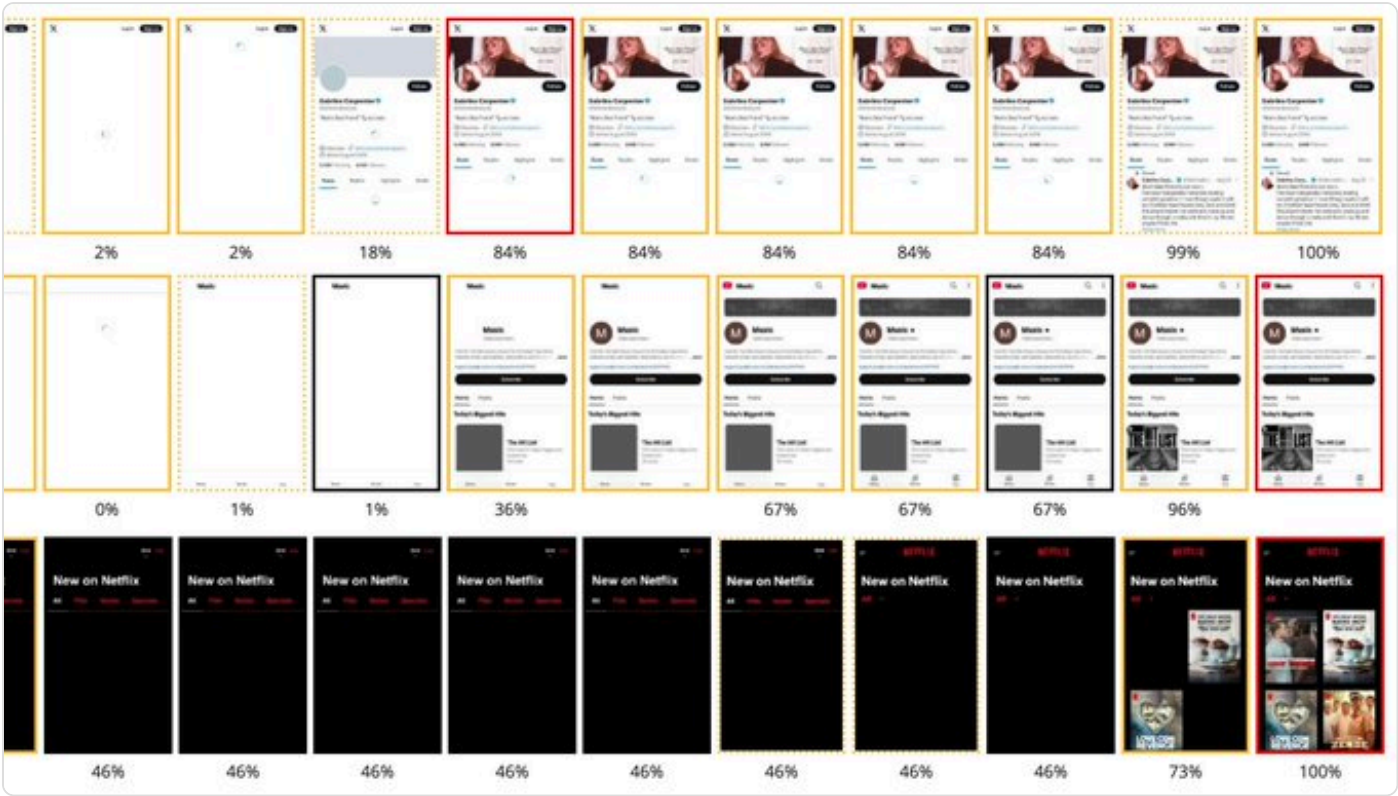
Even if an attacker finds a bug to run native code in the renderer, they then face the sandbox barrier — they'd need a separate exploit to break out to the system. This layered approach (site isolation + sandbox) is state-of-the-art for browser security.

Out-of-Process Iframes (OOPIF)

In Chrome's implementation, each frame of a page can potentially be backed by a different renderer process. The top-level frame and same-site frames share one process; cross-site frames use different processes. All those processes "cooperate" to render a single tab's content, coordinated by the browser process. The web platform (through specs like COOP/COEP, SharedArrayBuffer) is evolving with these constraints in mind after Spectre.

Site isolation does increase memory usage (10–20% overhead in some cases). Chrome mitigates this through "best-effort process consolidation" for same-site pages and limits on process spawning.

9. Comparing Chromium, Gecko, and WebKit



Comparing Chromium, Gecko, and WebKit — Illustration by Susie Lu

CSS / Style System

ENGINE	STYLE COMPUTATION	KEY DIFFERENCE
Blink (Chromium)	Single-threaded C++	Sequential computation with incremental invalidation
Gecko (Firefox)	Multi-threaded Rust (Stylo)	Parallel style computation across CPU cores via Quantum project (2017)
WebKit (Safari)	Single-threaded C++	Bytecode JIT for CSS selectors matching

Layout and Graphics

Firefox (Gecko) introduced WebRender as its compositor/rasterizer — now the default. WebRender (also Rust) takes the display list and renders it on the GPU directly, handling tessellating shapes, text, etc. This means Firefox can potentially animate more content at high performance because it redraws via GPU quick-

ly, akin to a game engine. Chrome's pipeline still rasterizes on CPU (for most content) then sends bitmaps to GPU.

Safari (WebKit) uses Core Animation layers (CALayer). Safari was early to move to GPU compositing (iPhone OS and Safari 4 in 2009 had hardware-accelerated compositing for CSS transforms).

JavaScript Engines

ENGINE	BROWSER	TIERS
V8	Chromium	Ignition → Sparkplug → Maglev → TurboFan (Turboshift)
SpiderMonkey	Firefox	Baseline Interpreter → Baseline JIT → WarpMonkey
JavaScriptCore	Safari	LLInt → Baseline → DFG → FTL (uses LLVM)

Multi-Process Model Differences

- **Chrome:** Each tab typically separate, site isolation at origin level, lots of processes (can be dozens).
- **Firefox:** Fewer processes by default (8 content processes handling all tabs). Tabs share content processes in a pool — lower memory, but one crash can take out multiple tabs.
- **Safari:** Likely one process per tab. On iOS, WKWebView definitely isolates each webview.

Rendering Pipeline Comparison

Chromium: Parse → Style → Layout → Paint → Layer tree → Raster → Composite

Gecko: Parse → Frame tree → Display list → WebRender scene → Composite

WebKit: Parse → Render tree → Graphics layers → Composite (via CoreAnimation)

Performance Trade-offs

Chrome was lauded for faster JS and overall performance due to multi-process and V8. Firefox with Quantum closed a lot of gaps, sometimes surpassing Chrome in graphics (WebRender can be very fast for complex pages). Safari often excels in graphics and low power usage on Apple hardware. Chrome has a reputation for high memory usage (all those processes); Firefox is more conservative; Safari is very memory efficient on iOS.

The View Transitions API reached Baseline Newly Available status in October 2025, with same-document transitions supported across Chrome 111+, Edge 111+, Firefox 133+, and Safari 18+. Cross-document transitions are supported in Chrome 126+, Edge 126+, and Safari 18.2+, with Firefox support still pending.

10. Conclusion and Further Reading

We've journeyed through the life of a web page inside a modern browser — from the moment a URL is entered, through networking and navigation, HTML parsing, styling, layout, painting, and JavaScript execution, all the way to the GPU putting pixels on the screen. Browsers are essentially mini operating systems: managing processes, threads, memory, and complex subsystems to ensure web content loads fast and runs securely.

Key Takeaways for Developers

- **Optimize network usage:** Fewer round trips and smaller files = faster start render. Leverage resource hints and efficient caching.
- **Structure HTML/CSS for efficiency:** A well-structured DOM and lean CSS help the parsing and style systems. Avoid very deep trees or overly complex selectors.
- **Batch DOM updates** to avoid repeated style/layout thrash.
- **Use compositing-friendly CSS for animations:** Animations of `transform` or `opacity` stay off the main thread and on the compositor, yielding smooth animations.
- **Mind the JS execution:** Long tasks block the main thread. Break up long operations and consider Web Workers for background tasks.
- **Embrace security features:** Use `iframe sandbox` or `rel=noopener` when appropriate.
- **DevTools is your friend:** The performance and network panels are gold mines for seeing exactly what the browser is doing.

Further Reading

- **Browser Engineering** — a great free book by Pavel Panchekha and Chris Harrelson
- **Chromium University** — free series of deep-dive videos into how Chromium works, including the excellent *Life of a Pixel* talk
- **Inside look at modern web browser** — Chrome team's multi-part series with diagrams
- **V8 Blog** — for learning about engine advances
- **Mozilla's Hacks Blog** — for WebRender internals and more

→ **Google Chrome at 17** — a history of our browser

Illustrations in this piece are by Susie Lu. Original article published on X by Addy Osmani, June 21, 2026. Formatted for off-line reading.

